

데이터베이스 (DataBase)

전북대학교 컴퓨터인공지능학부

이경수

13

CHAPTER

MongoDB Schema

- ❑ MongoDB 스키마의 개념
- ❑ MongoDB Design Pattern



MongoDB 스키마

Schema(스키마)의 개념

- MongoDB에서의 Schema(스키마)란?

- **스키마 (Schema)** 는 데이터베이스에서 데이터의 구조 (structure), 속성 (attributes), 제약조건 (constraints) 등을 정의하는 설계도와 같은 개념
- 전통적인 관계형 데이터베이스(RDBMS)에서는 고정된 스키마를 따르지만, MongoDB는 스키마리스(schema-less) 구조를 지원

- MongoDB에서의 스키마의 특징

- MongoDB는 유연한 문서 기반(Document-oriented) 구조이기 때문에 같은 컬렉션 안의 문서들이 서로 다른 필드를 가질 수 있음.
- 하지만, Mongoose 같은 ODM(Object Data Modeling) 라이브러리를 사용하면, Node.js와 같은 환경에서도 명시적인 스키마를 정의할 수 있음.

MongoDB Schema

Schema(스키마)의 개념

• MongoDB에서의 스키마의 특징

- MongoDB는 유연한 문서 기반(Document-oriented) 구조이기 때문에 같은 컬렉션 안의 문서들이 서로 다른 필드를 가질 수 있음.
- 하지만, Mongoose 같은 ODM(Object Data Modeling) 라이브러리를 사용하면, Node.js와 같은 환경에서도 명시적인 스키마를 정의할 수 있음.
- User라는 모델을 만들기 위해 name, age, email이라는 필드가 포함된 스키마를 정의

• 핵심 요약

- MongoDB는 기본적으로 스키마가 없는(NoSQL) 데이터베이스
- 하지만, 데이터 일관성과 검증을 위해 ODM을 통해 스키마 정의 가능
- 스키마 정의는 개발자의 의도와 구조화된 데이터 설계를 반영하는 방법

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: String,
  age: Number,
  email: String
});

const User = mongoose.model('User', userSchema);
```

MongoDB Schema

왜 스키마 디자인이 중요한가?

- 성능: 쿼리 효율성, 디스크 I/O
- 확장성: 수평 확장(Sharding), 데이터 파티셔닝
- 유지보수성: 일관된 데이터 구조로 코드 안정성 향상
- 읽기 vs 쓰기 최적화에 따라 설계 전략이 달라짐

MongoDB Schema

왜 스키마 디자인이 중요한가?

```
db.movie.insertOne(  
  {  
    _id: 1,  
    title: "Titanic",  
    year: 1997,  
    director: "James Cameron",  
    runtime: 194,  
    distributor: "Paramount Pictures",  
    languages: [ "English" ],  
    total_views: 3500  
  }  
)
```

```
db.theater.insertMany(  
  [  
    {  
      name: "Downtown Cinemas",  
      address: {  
        street: "2212 Taylor Street",  
        state: "NY"  
      },  
      movies: [  
        {  
          movie_id: 1,  
          title: "Titanic",  
          runtime: 194,  
          views: 1500  
        }  
      ]  
    },  
    {  
      name: "Midtown Theater",  
      address: {  
        street: "1232 5th Street",  
        state: "NY"  
      },  
      movies: [  
        {  
          movie_id: 1,  
          title: "Titanic",  
          runtime: 194,  
          views: 2000  
        }  
      ]  
    }  
  ]  
)
```


MongoDB Schema

데이터 모델링 개념

- 데이터 모델링(Data Modeling)은 데이터를 효율적이고 일관되게 저장하고 접근할 수 있도록 구조를 설계하는 과정
- MongoDB에서는 문서(Document) 단위로 데이터를 모델링하며, 관계형 데이터베이스의 정규화 접근과는 다른 방식이 필요함.
- 스키마 유연성: 필드와 데이터 타입이 고정되지 않음.
- 중첩 문서(Embedded Document)와 배열을 자유롭게 사용 가능.
- 관계 표현 방식이 전통적인 DBMS와 다름:
 - RDBMS: 테이블 + Foreign Key
 - MongoDB: 임베딩 vs 레퍼런싱

MongoDB Schema

데이터 모델링이 중요한 이유

- 애플리케이션 요구사항에 최적화된 설계를 가능하게 함
- 쿼리 성능 최적화와 데이터 일관성 유지에 직접적 영향
- 읽기/쓰기 비율, 데이터 크기, 확장 전략에 따라 모델링 전략이 달라짐

MongoDB Schema

데이터 모델링: 일대일 관계

```
// patron document
{
  _id: "joe",
  name: "Joe Bookreader"
}
// address document
{
  street: "123 Fake Street",
  city: "Faketon",
  state: "MA",
  zip: "12345"
}
```

```
{
  _id: "joe",
  name: "Joe Bookreader",
  address: {
    street: "123 Fake Street",
    city: "Faketon",
    state: "MA",
    zip: "12345"
  }
}
```

MongoDB Schema

데이터 모델링: 일대일 관계

```
{
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216,
  language: "English",
  publisher: {
    name: "O'Reilly Media",
    founded: 1980,
    location: "CA"
  }
}

{
  title: "50 Tips and Tricks for MongoDB Developer",
  author: "Kristina Chodorow",
  published_date: ISODate("2011-05-06"),
  pages: 68,
  language: "English",
  publisher: {
    name: "O'Reilly Media",
    founded: 1980,
    location: "CA"
  }
}
```

```
{
  _id: "oreilly",
  name: "O'Reilly Media",
  founded: 1980,
  location: "CA"
}

{
  _id: 123456789,
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216,
  language: "English",
  publisher_id: "oreilly"
}

{
  _id: 234567890,
  title: "50 Tips and Tricks for MongoDB Developer",
  author: "Kristina Chodorow",
  published_date: ISODate("2011-05-06"),
  pages: 68,
  language: "English",
  publisher_id: "oreilly"
}
```

MongoDB Schema

데이터 모델링: 일대다 관계

```
{
  _id: "book001",
  title: "Cell Biology",
  authors: [
    {
      author_id: "author124",
      name: "Ellie Smith"
    },
    {
      author_id: "author381",
      name: "John Palmer"
    }
  ]
}

{
  _id: "book002",
  title: "Organic Chemistry",
  authors: [
    {
      author_id: "author290",
      name: "Jane James"
    },
    {
      author_id: "author381",
      name: "John Palmer"
    }
  ]
}
```

MongoDB Schema

데이터 모델링: 일대다 관계

```
// patron document
{
  _id: "joe",
  name: "Joe Bookreader"
}

// address one
{
  street: "123 Fake Street",
  city: "Faketon",
  state: "MA",
  zip: "12345"
}

// address two
{
  street: "1 Some Other Street",
  city: "Boston",
  state: "MA",
  zip: "12345"
}
```

```
{
  _id: "joe",
  name: "Joe Bookreader",
  addresses: [
    {
      street: "123 Fake Street",
      city: "Faketon",
      state: "MA",
      zip: "12345"
    },
    {
      street: "1 Some Other Street",
      city: "Boston",
      state: "MA",
      zip: "12345"
    }
  ]
}
```



Design Pattern

MongoDB Schema

MongoDB 스키마 설계 패턴 개요

패턴 이름	설명	사용 예시
Embedding	문서 내에 다른 데이터 직접 포함	사용자 → 게시글 댓글 내장
Referencing	문서 간 참조 (ObjectId로 연결)	사용자 ↔ 게시글 ↔ 댓글 (분리형)
Bucket Pattern	여러 개의 이벤트/기록을 한 문서에 그룹화	센서 데이터 로그 등
Subset Pattern	자주 사용하는 필드만 따로 저장	게시글 목록 미리보기 등
Computed Pattern	계산된 필드를 미리 저장	조회수, 평균평점 등

Embedding Pattern

- 관련된 데이터를 하나의 문서에 중첩하여 저장하는 방식
- 조회 시 JOIN 없이 한 번의 read로 모든 관련 정보 접근 가능
- 읽기(Read)가 많고, 데이터 크기가 작거나 변경이 적을 때 유리
- 장점:
 - ⇒ 빠른 조회 (Join 불필요)
 - ⇒ 데이터 일관성 보장
- 단점:
 - ⇒ 중복 가능성 있음 (다른 문서에도 같은 구조가 있을 수 있음)
 - ⇒ 문서 크기 제한(16MB)을 초과할 수 있음

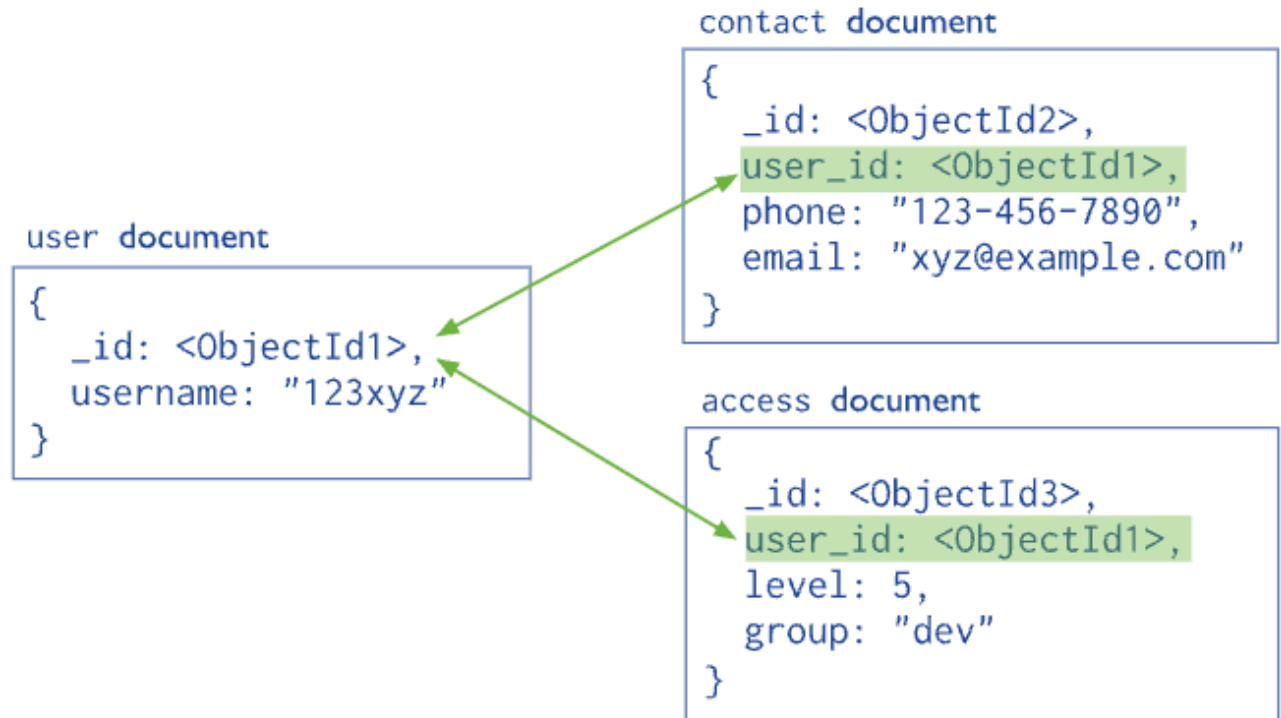
```
{
  _id: <ObjectId>,
  username: "123xyz",
  contact: {
    phone: "123-456-7890",
    email: "xyz@example.com"
  },
  access: {
    level: 5,
    group: "dev"
  }
}
```

Embedded sub-document

Embedded sub-document

Reference Pattern

- 관련 데이터를 다른 컬렉션에 분리하고, ObjectId를 참조(Reference)하여 연결하는 방식
- 관계형 DB의 Foreign Key 개념과 유사
- 쓰기/수정이 잦거나, 데이터가 커질 가능성이 있을 때 적합
- 장점:
 - ⇒ 데이터 중복 최소화
 - ⇒ 각 문서를 독립적으로 관리 및 수정 가능
- 단점:
 - ⇒ 조인 기능이 제한적이므로, 여러 번의 쿼리(fetch) 필요
 - ⇒ 복잡한 조회 시 성능 저하 가능



Bucket Pattern

- 고빈도 데이터(이벤트/로그 등)를 일정 기준으로 묶어 하나의 문서에 저장하는 방식
- 데이터 크기나 개수가 매우 많을 때 유용
- MongoDB의 문서 수를 줄이고, 인덱스 부담을 완화할 수 있음
- 주요 특징
 - "버킷(bucket)"은 시간, 사용자, 카테고리 등의 기준으로 그룹화
 - 반복되는 데이터 구조를 하나의 문서 배열로 묶어 저장
 - 데이터 조회 시 일부만 가져오거나 필터링할 수 있음

```
db.trades.insertMany([
  {
    "ticker" : "MDB",
    "customerId": 123,
    "type" : "buy",
    "quantity" : 419,
    "date" : ISODate("2023-10-26T15:47:03.434Z")
  },
  {
    "ticker" : "MDB",
    "customerId": 123,
    "type" : "sell",
    "quantity" : 29,
    "date" : ISODate("2023-10-30T09:32:57.765Z")
  },
  {
    "ticker" : "GOOG",
    "customerId": 456,
    "type" : "buy",
    "quantity" : 50,
    "date" : ISODate("2023-10-31T11:16:02.120Z")
  }
])
```

Bucket Pattern

- 주요 특징

- "버킷(bucket)"은 시간, 사용자, 카테고리 등의 기준으로 그룹화
- 반복되는 데이터 구조를 하나의 문서 배열로 묶어 저장
- 데이터 조회 시 일부만 가져오거나 필터링할 수 있음

- 사용 예시가 적합한 경우

- IoT 센서 데이터 수집 (초당 수십 건)
- 주식 거래 기록, 웹 방문 로그, 사용자 활동 히스토리
- 특정 고객의 다수 이벤트를 한 문서로 압축

```
db.trades.insertMany([
  {
    "ticker" : "MDB",
    "customerId": 123,
    "type" : "buy",
    "quantity" : 419,
    "date" : ISODate("2023-10-26T15:47:03.434Z")
  },
  {
    "ticker" : "MDB",
    "customerId": 123,
    "type" : "sell",
    "quantity" : 29,
    "date" : ISODate("2023-10-30T09:32:57.765Z")
  },
  {
    "ticker" : "GOOG",
    "customerId": 456,
    "type" : "buy",
    "quantity" : 50,
    "date" : ISODate("2023-10-31T11:16:02.120Z")
  }
])
```

Bucket Pattern

- 장점:

- 문서 수 감소 → 인덱스 크기 축소 → 성능 향상
- 저장 공간 최적화

- 단점:

- 문서 크기 제한(16MB)을 넘지 않도록 주의
- 개별 이벤트 단위의 조작이 어려움 (부분 업데이트 제한)

```
db.trades.insertMany([
  {
    "ticker" : "MDB",
    "customerId": 123,
    "type" : "buy",
    "quantity" : 419,
    "date" : ISODate("2023-10-26T15:47:03.434Z")
  },
  {
    "ticker" : "MDB",
    "customerId": 123,
    "type" : "sell",
    "quantity" : 29,
    "date" : ISODate("2023-10-30T09:32:57.765Z")
  },
  {
    "ticker" : "GOOG",
    "customerId": 456,
    "type" : "buy",
    "quantity" : 50,
    "date" : ISODate("2023-10-31T11:16:02.120Z")
  }
])
```

MongoDB Schema

Bucket Pattern

```
{
  "customerId": 123,
  "history": [
    {
      "type": "buy",
      "ticker": "MDB",
      "qty": 419,
      "date": ISODate("2023-10-26T15:47:03.434Z")
    },
    {
      "type": "sell",
      "ticker": "MDB",
      "qty": 29,
      "date": ISODate("2023-10-30T09:32:57.765Z")
    }
  ]
},
{
  "customerId": 456,
  "history": [
    {
      "type" : "buy",
      "ticker" : "GOOG",
      "quantity" : 50,
      "date" : ISODate("2023-10-31T11:16:02.120Z")
    }
  ]
}
```

```
db.trades.drop()

db.trades.insertMany(
[
  {
    "_id": "123_1698349623",
    "customerId": 123,
    "count": 2,
    "history": [
      {
        "type": "buy",
        "ticker": "MDB",
        "qty": 419,
        "date": ISODate("2023-10-26T15:47:03.434Z")
      },
      {
        "type": "sell",
        "ticker": "MDB",
        "qty": 29,
        "date": ISODate("2023-10-30T09:32:57.765Z")
      }
    ]
  },
  {
    "_id": "456_1698765362",
    "customerId": 456,
    "count": 1,
    "history": [
      {
        "type" : "buy",
        "ticker" : "GOOG",
        "quantity" : 50,
        "date" : ISODate("2023-10-31T11:16:02.120Z")
      }
    ]
  }
]
```

MongoDB Schema

Bucket Pattern

```
db.trades.updateOne( { "_id": /^123_/, "count": { $lt: 10 } },
  {
    "$push": {
      "history": {
        "type": "buy",
        "ticker": "MSFT",
        "qty": 42,
        "date": ISODate("2023-11-02T11:43:10")
      }
    },
    "$inc": { "count": 1 },
    "$setOnInsert": { "_id": "123_1698939791", "customerId": 123 }
  },
  { upsert: true }
)
```

```
[
  {
    _id: '123_1698349623',
    customerId: 123,
    count: 3,
    history: [
      {
        type: 'buy',
        ticker: 'MDB',
        qty: 419,
        date: ISODate("2023-10-26T15:47:03.434Z")
      },
      {
        type: 'sell',
        ticker: 'MDB',
        qty: 29,
        date: ISODate("2023-10-30T09:32:57.765Z")
      },
      {
        type: 'buy',
        ticker: 'MSFT',
        qty: 42,
        date: ISODate("2023-11-02T11:43:10.000Z")
      }
    ]
  },
  {
    _id: '456_1698765362',
    customerId: 456,
    count: 1,
    history: [
      {
        type: 'buy',
        ticker: 'GOOG',
        quantity: 50,
        date: ISODate("2023-10-31T11:16:02.120Z")
      }
    ]
  }
]
```

MongoDB Schema

Computed Pattern

- 계산된 결과 값(derived value)을 문서에 미리 저장(precomputed)하는 방식
- MongoDB의 Aggregation Pipeline 연산 비용을 줄이기 위해 사용
- 읽기(read) 성능을 향상시키고, 복잡한 쿼리 없이 데이터 제공 가능

```
db.screenings.insertMany( [  
  {  
    theater: "Alger Cinema",  
    location: "Lakeview, OR",  
    movie_title: "Lost in the Shadows",  
    movie_id: 1,  
    num_viewers: 344,  
    revenue: 3440  
  },  
  {  
    theater: "City Cinema",  
    location: "New York, NY",  
    movie_title: "Lost in the Shadows",  
    movie_id: 1,  
    num_viewers: 1496,  
    revenue: 22440  
  },  
])
```


MongoDB Schema

Computed Pattern

- 계산식 예: $\text{revenue} = \text{ticket_price} \times \text{num_viewers}$
- 정적/예측 가능한 계산 결과를 문서에 직접 포함시킴
- 읽기 성능 향상이 필요한 상황에 적합
- 조회수가 많은 리포트/대시보드 제공 (예: 총 판매액, 평균 평점)
- 검색결과 정렬용 필드 미리 계산 (예: 인기순, 할인율 등)
- 주기적 갱신만으로 충분한 경우 (실시간 정확성 불필요)

```
db.screenings.insertMany( [  
  {  
    theater: "Alger Cinema",  
    location: "Lakeview, OR",  
    movie_title: "Lost in the Shadows",  
    movie_id: 1,  
    num_viewers: 344,  
    revenue: 3440  
  },  
  {  
    theater: "City Cinema",  
    location: "New York, NY",  
    movie_title: "Lost in the Shadows",  
    movie_id: 1,  
    num_viewers: 1496,  
    revenue: 22440  
  },  
])
```

MongoDB Schema

Computed Pattern

```
db.screenings.insertMany( [  
  {  
    theater: "Alger Cinema",  
    location: "Lakeview, OR",  
    movie_title: "Lost in the Shadows",  
    movie_id: 1,  
    num_viewers: 344,  
    revenue: 3440  
  },  
  {  
    theater: "City Cinema",  
    location: "New York, NY",  
    movie_title: "Lost in the Shadows",  
    movie_id: 1,  
    num_viewers: 1496,  
    revenue: 22440  
  },  
]  
)
```

```
db.movies.insertOne(  
  {  
    _id: 1,  
    title: "Lost in the Shadows",  
    total_viewers: 1840,  
    total_revenue: 25880  
  }  
)  
  
db.screenings.insertOne(  
  {  
    theater: "Overland Park Cinema",  
    location: "Boise, ID",  
    movie_title: "Lost in the Shadows",  
    movie_id: 1,  
    num_viewers: 760,  
    revenue: 7600  
  }  
)
```

```
db.screenings.aggregate( [  
  {  
    $group: {  
      _id: "$movie_id",  
      total_viewers: {  
        $sum: "$num_viewers"  
      },  
      total_revenue: {  
        $sum: "$revenue"  
      }  
    },  
  },  
  {  
    $merge: {  
      into: { db: "test", coll: "movies" },  
      on: "_id",  
      whenMatched: "merge"  
    }  
  }  
]  
)
```

Thanks!

질문은 여기에...

ksl@jbnu.ac.kr

